

大家在看

基于SpringBoot的OA系统的
686

 **徐卖狼**
博客等级  码
Effective C++ 条款11：在o
每一个成分
208 220 4
Python效率革命：用NumP
处理 原创 828 点赞 收
Effective C++ 条款11：在o
关注



热门文章

running
项目启动 zookeeper 报错
java.net.ConnectException
2022年 5篇 20
refused: no further informa
路 2022年 09篇 201
MaxPermSize 4G
2018年 48篇
stack size e:
析 34837



分类专栏

 macOS

 docker

 mq

 Java

[展开全部](#)



最新评论

谷歌浏览器插件之 -- Quick
猪全校: 安装好了 要安装里

目录

- jstack 命令
- 什么是jstack
- jstack命令
- jstack实战操作

收起

原创 徐卖狼 于 2020-12-04 09:54:31 发布 阅读量1.6w 收藏 28 点赞数 10
分类专栏: mysql



GitCode 开源社区 文章已被社区收录



mysql 专栏收录该内容

jstack 命令

- 什么是jstack
- jstack命令
- jstack实战操作

什么是jstack

jstack是用于生成java虚拟机当前时刻的线程快照。线程快照是当前java虚拟机内存每一条线程正在执行的方法堆栈的集合，出现长时间停顿的原因

如果出现 **死锁**，死循环，请求外部资源出现长时间等待等，线程出现停顿的时候，通过jstack来查看各个线程调用堆栈，待什么资源，如果java程序崩溃生成core文件，jstack工具可以用来获取core文件的java，stack和native stack的信息，从而和在程

程序触发生成的问题，另外，jstack工具还可以附属到正在运行的java程序中看到当时运行的java程序的java stack和native：呈现bug的状态，jstack是非常有用的。

jstack命令

```
$ jstack
Usage:
  jstack [-l] <pid>
           (to connect to running process)
  jstack -F [-m] [-l] <pid>
           (to connect to a hung process)
  jstack [-m] [-l] <executable> <core>
           (to connect to a core file)
  jstack [-m] [-l] [server_id@]<remote server IP or hostname>
           (to connect to a remote debug server)

Options:
  -F to force a thread dump. Use when jstack <pid> does not respond (
  -m to print both java and native frames (mixed mode)
  -l long listing. Prints additional information about locks
  -h or -help to print this help message
```

参数说明：

- -F 当 'jstack [-l] pid' 没有响应的时候强制打印栈信息，如果直接jstack无响应时，用于强制jstack，一般情况下无需使用
- -l 长列表 打印关于锁定附加信息，例如属于：java.util.concurrent的ownable synchronizers列表会使jvm停顿的长久的多
- -m 打印java和native c/c++ 框架的所有栈信息.可以打印JVM的堆栈,显示上Native的栈帧，一般应用排查不需要使用

执行命令

```
jstack -m 12905
```

大家在看

基于SpringBoot的OA系统的
686



徐卖狼
博客等级 4
Effective C++ 条款11：在o

每一个成分

208 220 4
Python效率革命：用NumP
处理 828 点赞 收

Effective C++ 条款11：在o
关注



热门文章

running
项目启动 zookeeper 报错
java.net.ConnectException
2022年 5篇 20
refused: no further informa
路99分 09篇9 201
MaxPermSize stack size e:
析 34837

解决 maven clean 报错 Pro
29942

redis 基于SETNX和EXPIRE
redis 分布式锁 24290

idea 添加格式化json插件G:
速解析第三方返回json数据

分类专栏

- macOS
- docker
- mq
- Java

展开全部

上一篇: zookeeper数据与

下一篇: 谷歌浏览器插件之

最新评论

谷歌浏览器插件之 -- Quick
猪全校: 安装好了 要安装里

目录

jstack 命令

什么是jstack

jstack命令

jstack实战操作

收起

```
ERROR: attach: task_for_pid(12905) failed: '(os/kern) failure' (5)
Error attaching to process: sun.jvm.hotspot.debugger.DebuggerException: Can't attach to the process. Could be caused by an incorrect
of privileges.
sun.jvm.hotspot.debugger.DebuggerException: sun.jvm.hotspot.debugger.DebuggerException: Can't attach to the process
ect pid or lack of privileges.
    at sun.jvm.hotspot.debugger.bsd.BsdDebuggerLocal$BsdDebuggerLocalWorkerThread.execute(BsdDebuggerLocal.java:144)
    at sun.jvm.hotspot.debugger.bsd.BsdDebuggerLocal.attach(BsdDebuggerLocal.java:287)
    at sun.jvm.hotspot.HotSpotAgent.attachDebugger(HotSpotAgent.java:671)
    at sun.jvm.hotspot.HotSpotAgent.setupDebuggerDarwin(HotSpotAgent.java:659)
    at sun.jvm.hotspot.HotSpotAgent.setupDebugger(HotSpotAgent.java:341)
    at sun.jvm.hotspot.HotSpotAgent.go(HotSpotAgent.java:304)
    at sun.jvm.hotspot.HotSpotAgent.attach(HotSpotAgent.java:140)
    at sun.jvm.hotspot.tools.Tool.start(Tool.java:185)
    at sun.jvm.hotspot.tools.Tool.execute(Tool.java:118)
    at sun.jvm.hotspot.tools.JStack.main(JStack.java:92)
    at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
    at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
    at java.lang.reflect.Method.invoke(Method.java:498)
    at sun.tools.jstack.JStack.runJStackTool(JStack.java:140)
    at sun.tools.jstack.JStack.main(JStack.java:106)
Caused by: sun.jvm.hotspot.debugger.DebuggerException: Can't attach to the process. Could be caused by an incorrect
    at sun.jvm.hotspot.debugger.bsd.BsdDebuggerLocal.attach0(Native Method)
    at sun.jvm.hotspot.debugger.bsd.BsdDebuggerLocal.access$100(BsdDebuggerLocal.java:65)
    at sun.jvm.hotspot.debugger.bsd.BsdDebuggerLocal$1AttachTask.doit(BsdDebuggerLocal.java:278)
    at sun.jvm.hotspot.debugger.bsd.BsdDebuggerLocal$BsdDebuggerLocalWorkerThread.run(BsdDebuggerLocal.java:144)
```

科普一下线程状态：

NEW	未启动的。不会出现在Dump中
RUNNABLE	在虚拟机内执行的
BLOCKED	受阻塞并等待监视器锁
WAITING	无限期待另一个线程执行特定操作。
TIMED_WAITING	有时限的等待另一个线程的特定操作
TERMINATED	已退出的

jstack实战操作

现在假设一个场景，在几乎没有什么调用的情况下，服务器的cpu的使用率一直居高不下非常有可能是程序中出现死循环导致。如果cpu占用率过高，除了死循环以外，常见的还有，内存泄漏，导致大量的full GC

jstack实战之cpu 占用过高----测试代码

```
1  /**
2   * @Author: corey
3   * @Date: 2020/8/5 14:25
4   * @Description: jstack 测试代码
5   */
6
7  public class JstackTest {
8      private static Executor executor = Executors.newFixedThreadPool(5);
9
10     private static final Object lock = new Object();
11
12     public static void main(String[] args) {
13         MyRunnableImpl myRunnable = new MyRunnableImpl();
14         MyRunnableImpl myRunnable1 = new MyRunnableImpl();
15         executor.execute(myRunnable);
16         executor.execute(myRunnable1);
17     }
18
19
20     static class MyRunnableImpl implements Runnable{
21
22         @Override
23         public void run() {
24             synchronized (lock){
25                 //死循环
```

大家在看

基于SpringBoot的OA系统的9686

徐卖狼 及其使用场景(一) 博客等级 码

Effective C++ 条款11: 在o

Python效率革命：用NumP 208 220 4: 原创 828 点赞 收

Effective C++ 条款11: 在o 关注



热门文章

running 项目启动 zookeeper 报错 java.net.ConnectException 2022年 5篇 20% refused: no further informa 2022年 09篇 201

解决 maven clean 报错 Pro 29942

redis 基于SETNX和EXPIRE redis 分布式锁 24290

idea 添加格式化json插件G: 速解析第三方返回json数据

idea 添加格式化json插件G: 速解析第三方返回json数据

分类专栏

- macOS
- docker
- mq
- Java

展开全部

上一篇: zookeeper数据与

下一篇: 谷歌浏览器插件之

最新评论

谷歌浏览器插件之 - Quick 猪全校: 安装好了 要安装里

目录

jstack 命令

什么是jstack

jstack命令

jstack实战操作

收起



```
29     private void calculate(){
30         int i = 0;
31         while (true){
32             i++;
33         }
34     }
35 }
36 }
```

收起 ^

PID	COMMAND	%CPU	TIME	#TH	#WQ	#PORT	MEM	PURG	CMPS	PGRP	PPID	STATE	BOOSTS	%CPU
1093	java	99.3	00:21.12	27/1	1	93	14M	0B	0B	1568	1568	running	*0[1]	0.00.00
4067	top	12.7	00:14.48	1/1	0	33	5664K	0B	0B	4067	3231	running	*0[1]	0.00
236	WindowServer	5.0	47:57.43	10	4	2168-	812M+	11M-	45M-	236	1	sleeping	*0[1]	1.40
206	coreaudiod	4.9	32:55.86	8	2	361	28M	0B	3836K	206	1	sleeping	*0[1]	0.00
1191	Google Chrom	4.7	05:29.73	10	1	401+	403M+	65M-	37M	1179	1179	sleeping	*0[3]	0.00
776	Terminal	3.1	00:32.42	7	2	303+	93M+	21M	7640K	776	1	sleeping	*0[7839]	0.43
713	WeChat	2.6	11:40.32	44	17	5580-	400M-	22M-	71M	713	1	sleeping	*108[1620]	0.01
0	kernel_task	2.5	20:16.22	208/8	0	0	328M-	0B	0B	0	0	running	0[0]	0.00
1449	Google Chrom	2.4	16:25.85	25	1	476+	335M-	0B	49M	1179	1179	sleeping	*0[8]	0.00
286	TouchBarServ	2.3	01:31.72	8	4	763+	26M+	0B-	4200K	286	1	sleeping	*0[1]	0.01
1568	idea	1.8	40:29.05	85	1	565	2578M-	6500K	695M	1568	1	sleeping	*0[6005+]	0.00
1621	Google Chrom	1.8	02:04.34	15	1	338+	254M+	0B	35M-	1179	1179	sleeping	*0[4]	0.00

使用top命令可以看到，cpu的占用到99%了,使用 jps命令查看当前java 程序运行的有哪些。

```
[ $ jps
1568
4274 Launcher
4275 JstackTest
4344 Jps
4061 KotlinCompileDaemon
```

然后使用 jstack 4275 命令查看当前程序出现死循环的是在哪一行

```
pool-1-thread-1 #13 prio=5 os_prio=31 tid=0x00007fffd11875000 nid=0x5903 runnable [0x00007fffd11875000]
  java.lang.Thread.State: RUNNABLE
    at com.itcorey.interview.jstack.JstackTest$MyRunnableImpl.calculate(JstackTest.java:31)
    at com.itcorey.interview.jstack.JstackTest$MyRunnableImpl.run(JstackTest.java:31)
    - locked <0x0000000076ae302d0> (a java.lang.Object)
    at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149)
    at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
    at java.lang.Thread.run(Thread.java:748)
```

如下上图，JstackTest 程序的 31行

```
java.lang.Thread.State: BLOCKED (on object monitor)
  at JstackTest$MyRunnableImpl.run(JstackTest.java:32)
  - waiting to lock <0x00000000f145ef38> (a java.lang.Object)
  at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149)
  at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
  at java.lang.Thread.run(Thread.java:748)

"pool-1-thread-1" #9 prio=5 os_prio=0 tid=0x00007ff1580ee000 nid=0x7b9 runnable [0x00007ff1580ee000]
  java.lang.Thread.State: RUNNABLE
    at JstackTest$MyRunnableImpl.calculate(JstackTest.java:39)
    at JstackTest$MyRunnableImpl.run(JstackTest.java:32)
    - locked <0x00000000f145ef38> (a java.lang.Object)
    at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149)
    at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
    at java.lang.Thread.run(Thread.java:748)
```

①线程名。

②线程优先级。

③一个地址(该地址是什么地址，存疑)。

④线程十六进制id。

大家在看

基于SpringBoot的OA系统创
686

徐卖狼
博客等级 码
Effective C++ 第11章：在o
每一个成分

208 220 4
Python效率革命：用NumP
处理 828 点赞 收

Effective C++ 条款11：在o
关注



热门文章

running
项目启动 zookeeper 报错
java.net.ConnectException
2022年 5篇 20
refused: no further informa
路99年 09篇9 201

MaxHeapSize stack size e:
析 34837

解决 maven clean 报错 Pro
29942

redis 基于SETNX和EXPIRE
redis 分布式锁 24290

idea 添加格式化json插件G:
速解析第三方返回json数据

分类专栏

MacOS
docker
mq
Java

展开全部 ^

上一篇：zookeeper数据与

下一篇：谷歌浏览器插件之

最新评论

谷歌浏览器插件之 -- Quick
猪全校：安装好了 要安装里

目录

jstack 命令

什么是jstack

jstack命令

jstack实战操作

收起 ^



⑥线程当前所处方法。

⑦该箭头表示线程的执行历程。

jstack实战之死锁线程的定位-----测试代码

java

```
1 /**
2  * @Author: corey
3  * @Date: 2020/8/5 15:01
4  * @Description: 死锁线程的定位
5  */
6 public class JstackLockTest {
7
8     private static Executor executor = Executors.newFixedThreadPool(5);
9 }
```

展开

输出如下:

```
Connected to the target VM, address: '127.0.0.1:60376'
pool-1-thread-1获得了锁LockA
pool-1-thread-2获得了锁lockB
```

如上图所示没有继续输出;继续排查

使用 jps 命令查看

```
$ jps
1568
4545 Jps
4506 Launcher
4507 JstackLockTest
4061 KotlinCompileDaemon
```

使用jstack pid指令,查看指定进程的堆栈信息,观察并定位到死锁线程。

```
"pool-1-thread-2" #14 prio=5 os_prio=31 tid=0x00007f8c8a862800 nid=0x5b03 waiting for monitor entr
  java.lang.Thread.State: BLOCKED (on object monitor)
    at com.itcorey.interview.jstack.JstackLockTest$MyRunnableImplTwo.run(JstackLockTest.java:5
    - waiting to lock <0x0000000076ae309a0> (a java.lang.Object)
    - locked <0x0000000076ae309b0> (a java.lang.Object)
    at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149)
    at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
    at java.lang.Thread.run(Thread.java:748)

"pool-1-thread-1" #13 prio=5 os_prio=31 tid=0x00007f8cbb041000 nid=0x5903 waiting for monitor entr
  java.lang.Thread.State: BLOCKED (on object monitor)
    at com.itcorey.interview.jstack.JstackLockTest$MyRunnableImplOne.run(JstackLockTest.java:3
    - waiting to lock <0x0000000076ae309b0> (a java.lang.Object)
    - locked <0x0000000076ae309a0> (a java.lang.Object)
    at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149)
    at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
    at java.lang.Thread.run(Thread.java:748)
```

大家在看

基于SpringBoot的OA系统的
686

徐卖狼
博客等级 码
Effective C++ 第2版 第2版

每一个成分

208 220 4
Python效率革命: 用NumP
处理 828 点赞 收

Effective C++ 条款11: 在o
关注



热门文章

running
项目启动 zookeeper 报错
java.net.ConnectException
2022年 5篇 20
refused: no further informa
路90年 09篇 201

2018年 48篇
析 34837

解决 maven clean 报错 Pro
29942

redis 基于SETNX和EXPIRE
redis 分布式锁 24290

idea 添加格式化json插件G:
速解析第三方返回json数据

分类专栏

MacOS
docker
mq
Java

展开全部

上一篇: zookeeper数据与

下一篇: 谷歌浏览器插件之

最新评论

谷歌浏览器插件之 - Quick
猪全校: 安装好了 要安装里

目录

jstack 命令

什么是jstack

jstack命令

jstack实战操作

收起

第二个thread所示:

pool-1-thread-1 已经获取到了地址为<0x000000076ae309b0>的地址, 正在BLOCKED等待地址为 <0x000000076ae309a0

上述分析过程是对死锁和cpu 超高的情况下的场景分析, 我们可以通过诸如【统一获取锁的顺序(线程按照一定的顺序加锁); 试获, 取锁的时候加上一定的时限, 超过时限则放弃对该锁的请求, 并释放自己占有的锁)】、【死锁检测】等手段预防死锁, 没法解开了, 只能停掉程序, 修复bug后再启动服务了。

参考: <https://www.cnblogs.com/taiguoya/p/9470861.html>

https://blog.csdn.net/justry_deng/article/details/90447410

大家在看

基于SpringBoot的OA系统的686

徐卖狼 及其使用场景(一) Effective C++ 第11章: 每一个成分

208 Python效率革命: 用NumPy处理数据 828 220 4: 点赞 收藏

Effective C++ 条款11: 在o 关注

【JAVA】49. 死锁检测与诊断: Jstack与可视化工具

摘要: 本文详细解析了死锁的四大必要条件(互斥、请求和保持、不剥夺、循环等待), 介绍了Jstack工具检测死锁的方法及输出分析, 推荐

012-JDK可视化监控工具-jstack

一、概述 jstack是java虚拟机自带的一种堆栈跟踪工具。jstack用于打印出给定的java进程ID或corefile或远程调试服务的Java堆栈信息,

使用java的jstack排查程序运行情况_windows jstack

1、jstack介绍 jstack(Java Virtual Machine Stack Trace)是JDK提供的一个可以生成Java虚拟机当前时刻的线程快照信息的命令行工具。2、

jstat使用详解_jstack命令的详细用法

通过top或ps命令找到高 CPU 占用的 Java 进程 PID。执行jstack命令: jstack -l<pid>>thread_dump.txt AI写代码bash 1 在输出中寻找处于RUN

jstack -l pid 内容解析

其实这些内容有个名词叫做 JVM内部线程 下面我们对这些功能挨个解说 JDK 版本: Java HotSpot(TM) 64-Bit Server VM (build 24.51-

原来jdk自带了这么好玩的工具 > 使用 jstack定位死循环

什么是jstack jstack的主要作用是查看或者导出java线程的堆栈信息(快照); 用于堆栈跟踪, 当我们使用jstack命令时, 它会将指定进程内的所

jstack定位线程堆栈信息【案例汇总】_jstack 堆栈 分析

使用jstack [pid] 输出当前进程的堆栈信息。主要有两种使用方式,如下: 控制台输出堆栈信息 jstack pid,示例: 图-3 将堆栈信息输出到执行文件 j

Java使用jstack对进程CPU占用过高排查_jstack pid lgrep tid -a-CSDN...

printf "%x\n"线程tid #\n用于换行 这里做这一步是因为jstack 进程ID的内容显示的线程信息中线程ID nid是16进制的,才能grep查找。 4.打印线

JAVA面试题分享五百一十六: jstack 命令的使用和问题排查分析思路

企业级的Java程序中, java程序出现性能低下甚至直接崩溃时, 就不能像解决业务bug那样, 从行为表现上来修复问题了。性能问题一般是jv

使用 jstack 命令排查问题

jstack 命令什么是jstackjstack命令jstack实战操作 什么是jstack jstack是用于生成java虚拟机当前时刻的线程快照。线程快照是当前java虚拟

JVM之jstack命令解析_jstack 堆栈 分析

jstack全称叫JavaStackTrace,Java的堆栈跟踪工具,用于生成java虚拟机当前时刻的线程快照。 功能主要有两个如下 分析死锁; 分析CPU过高

原来jdk自带了这么好玩的工具 -- 使用 jstack定位死循环_jstack查看...

jstack [options] options 参数说明 jstack的option参数并不多,真正用到的也就三个;接下来我们一个个介绍一下 -F:当线程挂起(Suspended)时,

jstack命令(Java Stack Trace)

jstack命令(Java Stack Trace) 2012-03-02 14:12 fenglibing fenglibing的博客我要评论(1)字号: TIT jstack用于打印出给定的java进程ID或core

jstack命令排查性能问题

使用top -Hp命令查看某进程里面, 资源占用率高的线程id。通过Java自带的jstack命令查看线程id中锁死的代码行。 排查步骤 找到出现问题

Java程序员必备:jstack命令解析_java jstack

jstack [option] pid jstack [option] executable core jstack [option] [server-id@]remote-hostname-or-IP executable Java executable from wl

jstack分析cpu占用高应用_jstack cpu=

第三步:jstack输出堆栈信息:jstack pid 进行查看输出到临时文件 jstack pid | grep -A 50 tid-16进制> temp.jstack(pid 替换为查找到的pid数字,ti

利用工具JStack排查: 死锁问题和CPU100%问题

无论是再面试过程中还是再实际项目开发当中我们都可能遇到这两个问题。我之前有同学面试这两个问题都有问道过。哈哈哈。所以我绝

jstack-查找死锁, 线程堆栈分析工具

jstack是jdk自带的线程堆栈分析工具, 使用该命令可以查看或导出 java 应用程序中线程堆栈信息。 jstack用于生成java虚拟机当前时刻的线

JVM 性能调优监控工具之jstack

示例:jstack主要用来查看某个Java进程内的线程堆栈信息。语法格式如下。 jstack pid AI写代码java 运行 1 二、使用步骤 1.查询服务器pid 代

线上问题排查思路总结

前言: 本文总结一些常见的线上问题和对应的排查思路, 工具。对于线上问题, 我们必须记住一个原则: 尽快恢复服务, 消除影响。不管出



热门文章

running 项目启动 zookeeper 报错 java.net.ConnectException 2022年 5篇 202 refused: no further informa 路99年 09篇9 201

MaxHeapMemorySize stack size e: 析 34837

解决 maven clean 报错 Pro 29942

redis 基于SETNX和EXPIRE redis 分布式锁 24290

idea 添加格式化json插件G: 速解析第三方返回json数据

分类专栏

- MacOS
- docker
- mq
- Java

展开全部

上一篇: zookeeper数据与

下一篇: 谷歌浏览器插件之

最新评论

谷歌浏览器插件之 - Quick 猪全校: 安装好了 要安装里

目录

jstack 命令

什么是jstack

jstack命令

jstack实战操作

收起

java排查故障思路

线上的故障无非就是四种，cpu,磁盘，内存，网络。出现的大多数问题都不仅仅在一个层面上面，基本出现问题了就是df,free,top 三连，然后

通过jstack日志分析和问题排查 热门推荐

转自：http://www.ccblog.cn/84.htm 前段事件公司出现了一个严重故障（调用dubbo服务一直处于超时，整个服务系统发生雪崩，系统彻底宕

记一次利用jstack排查错误的过程

一直知道jstack这个命令，却没有怎么真正使用过，正好项目上碰到了一些问题，利用jstack辅助定位到了问题。下面对整个过程进行一个记

多线程——synchronized 为线程提供安全 （同步）

package thread; //不安全的买票 public class ThreeUnsafeCasesTicket { public static void main(String[] args) { BuyTicket buyTicket = new Bu

jstack

package com.atguigu.jstack; /** * 演示线程：TIMED_WAITING * * @author shkstart * @create 15:28 */ public class TreadSleepTest { public

帮我封装个微信小程序日期时间组件

好的，我可以为您提供一个微信小程序日期时间组件的封装。以下是示例代码： ``javascript Component({ properties: { value: { type: String, v

大家在看

基于SpringBoot的OA系统的
686

多线程及其使用场景(一)

Effective C++ 条款12：复
每一个成分

Python效率革命：用NumP
处理 828

Effective C++ 条款11：在o
“自我赋值”

最新文章

实现网页内容可编辑

接口调试工具 -谷歌插件 AI

单元测试异常: Command li
Shorten command line for
XXXXXXXTest.rmLogRecoi
running

2022年 5篇 202

2020年 109篇 201

2018年 48篇

目录

jstack 命令

什么是jstack

jstack命令

jstack实战操作

收起 ^